
Partial Reproduction of Self-Training with Pseudo-Label Scorer for Aspect Sentiment Quad Prediction

Anonymous authors
Paper under double-blind review

Abstract

1 Aspect-based sentiment analysis (ABSA) has evolved to the task of aspect
2 sentiment quadruple prediction (ASQP), which requires extracting aspect
3 terms, aspect categories, opinion terms, and sentiment polarities simulta-
4 neously. Despite its practical value, ASQP suffers from a severe scarcity
5 of labeled data due to the high cost and difficulty of fine-grained anno-
6 tation. This report presents our partial reproduction of the ACL 2024
7 paper “Self-Training with Pseudo-Label Scorer for Aspect Sentiment Quad
8 Prediction” (Zhang et al., 2024) based on the T5 framework. Following
9 the paper’s pipeline, we train a base quadruple generation model, generate
10 pseudo-labels for unlabeled restaurant reviews, and then train a scorer on
11 labeled data and a comparison dataset to filter low-quality pseudo-labels.
12 We further test reranking on beam search candidates during inference. On
13 ACOS-Rest and ASQP-Rest16, our reproduction confirms that self-training
14 without reranking improves over the baseline, while reranking can reduce
15 performance in our setting.

16 1 Introduction

17 Aspect-based sentiment analysis (ABSA) aims to extract fine-grained sentiment information
18 from user-generated content. The task has continuously evolved: from early subtasks such as
19 aspect term extraction, aspect category classification, and sentiment polarity classification,
20 to joint aspect-opinion extraction, aspect sentiment triplet extraction (ASTE), and finally to
21 the most challenging formulation—aspect sentiment quadruple prediction (ASQP). ASQP
22 requires predicting a quadruple consisting of aspect term, aspect category, opinion term,
23 and sentiment polarity. In this project, we focus on a partial reproduction of the ACL 2024
24 ASQP work and use the presentation outline to organize the report around background,
25 method, and experimental results.

26 One critical bottleneck for ASQP is the scarcity of labeled data. Because the task involves
27 four fine-grained components, manual annotation is both expensive and difficult. Publicly
28 available datasets are therefore small in scale, which limits the performance of supervised
29 models. Recent works have attempted to mitigate data scarcity via data augmentation or
30 distant supervision, but these approaches either introduce noise or require external knowl-
31 edge bases.

32 Following the reproduced method (Zhang et al., 2024), we use T5 as the backbone and
33 implement a three-stage self-training pipeline: first train a base quadruple generator on
34 labeled data, then generate pseudo-labels for unlabeled data, and finally train a scorer to
35 filter pseudo-labels and optionally rerank beam outputs. This setup matches the workflow
36 shown in the presentation and keeps the implementation scope focused on the restaurant-
37 domain experiments.

38 A key component of the reproduced method is the scoring model. Unlike discriminative
39 scorers that only estimate a single score, the scorer is a generative model that outputs the
40 conditional probability of a pseudo-label given the input sentence. This allows token-level
41 evaluation and better captures the interaction between the sentence and the quadruple. We
42 also compare ranking objectives and keep the listwise loss as the default, consistent with
43 the paper’s findings.

Experimental results on ACOS-Rest and ASQP-Rest16 show that our reproduction improves F1 over the baseline before reranking, but the reranking step does not help in our setting. We therefore analyze the effect of sample quantity, filtering interval, and ranking loss to explain this discrepancy.

The main contributions of this paper are:

- A novel pseudo-label scoring framework for ASQP that uses a generative scorer with a listwise loss.
- A self-training pipeline that effectively utilizes unlabeled data through score-based filtering.
- A partial reproduction of the ACL 2024 ASQP method (Zhang et al., 2024), with experiments focused on the restaurant-domain settings shown in the presentation.

2 Related work

Aspect Sentiment Quadruple Prediction. ASQP is a well-established task in aspect-based sentiment analysis. Subsequent works have explored various architectures, including sequence-to-sequence models and table-filling approaches. However, most existing methods rely on relatively small labeled datasets, limiting generalization.

Semi-supervised and Self-training for ABSA. Several studies have applied self-training to ABSA subtasks, using pseudo-labeling for aspect term extraction or distant supervision for sentiment classification. For ASQP, the multi-label and structured output nature makes pseudo-label noise more severe. Our work addresses this by learning a specialized scorer to filter noisy pseudo-labels, following the approach of Zhang et al. (2024).

Generative Models for Scoring. Generative scoring has been explored in machine translation and text summarization. Unlike discriminative models that output a scalar score, generative scorers compute conditional probabilities of the target given the source, which can capture fine-grained alignment. We adapt this idea to the ASQP task.

3 Methodology

3.1 Problem definition

Given a sentence $x = (w_1, \dots, w_n)$, the goal of ASQP is to generate a set of quadruples $\mathcal{Q} = \{(a_t, a_c, o_t, s)\}$, where a_t is the aspect term (a contiguous span in x), a_c is the aspect category (from a predefined set), o_t is the opinion term (a span in x), and $s \in \{\text{positive, negative, neutral}\}$ is the sentiment polarity. We follow the standard formulation of ASQP as a text-to-text generation task: the input is x , and the output is a linearized sequence of quadruples.

3.2 Base quadruple generation model

We adopt T5-base as the backbone generator. Let $D_L = \{(x_i, y_i)\}_{i=1}^N$ be the labeled dataset, where y_i is the linearized quadruple sequence. The model is trained with standard maximum likelihood estimation:

$$\mathcal{L}_{\text{gen}} = - \sum_{(x,y) \in D_L} \log p(y|x) = - \sum_{(x,y) \in D_L} \sum_{t=1}^{|y|} \log p(y_t | y_{<t}, x).$$

After training, we obtain a base generator G_{base} .

82 3.3 Pseudo-label generation

83 Given a large unlabeled dataset $D_U = \{x_j\}_{j=1}^M$, we use G_{base} to generate pseudo-labels
 84 via beam search. For each input x_j , we produce K candidate outputs $\{\hat{y}_j^{(k)}\}_{k=1}^K$ with their
 85 associated log-probabilities. The candidate with the highest probability is selected as the
 86 pseudo-label for filtering.

87 3.4 Training the scorer model

88 The scorer is designed to evaluate the quality of a pseudo-label $(x, \hat{y})$. Unlike discriminative
 89 scorers that output a single score, our scorer is a generative model (T5-large) that computes
 90 the conditional probability $p(\hat{y}|x)$. This probability serves as the score:

$$\text{score}(x, \hat{y}) \propto p(\hat{y}|x) = \prod_{t=1}^{|\hat{y}|} p(\hat{y}_t | \hat{y}_{<t}, x).$$

91 To train the scorer, we construct two types of data:

- 92 1. Labeled data D_L : Each (x, y) is a positive example where y is the ground-truth
 93 quadruple.
- 94 2. Comparison data D_{COMP} : For each input x in D_L , we generate R candidate out-
 95 puts from G_{base} . The ground-truth y is treated as the positive candidate y_p , and
 96 the remaining $R - 1$ candidates are considered negative samples y_n (some may be
 97 partially correct, but we treat all non-perfect outputs as negatives for training the
 98 ranker).

99 The scorer is trained with a loss that combines listwise ranking and language modeling
 100 objectives:

$$\mathcal{L} = \mathcal{L}_1 + \alpha \mathcal{L}_2,$$

101 where $\mathcal{L}_1$ is the listwise loss for comparison data, and $\mathcal{L}_2$ is the standard negative log-
 102 likelihood for positive examples.

103 For a given input x and a positive pseudo-label y_p with negative samples $\{y_n^{(1)}, \dots, y_n^{(m)}\}$,
 104 the listwise loss is defined as:

$$\mathcal{L}_{\text{LIST}} = -\log \frac{p(y_p|x)}{Z}, \quad Z = p(y_p|x) + \sum_n p(y_n^{(n)}|x).$$

105 Then:

$$\begin{aligned} \mathcal{L}_1 &= \mathbb{E}_{(x,y) \sim D_{\text{COMP}}} \mathcal{L}_{\text{LIST}}(x, y), \\ \mathcal{L}_2 &= \mathbb{E}_{(x,y_p) \sim D_{\text{COMP}} \cup D_L} -\log p(y_p|x). \end{aligned}$$

107 The hyperparameter α balances the two terms. Intuitively, $\mathcal{L}_1$ forces the model to assign
 108 higher probability to the positive candidate than to negatives, directly optimizing ranking
 109 quality, while $\mathcal{L}_2$ ensures the model does not forget the exact likelihood of correct quadruples.

110 3.5 Filtering and retraining

111 Using the trained scorer S , we filter the pseudo-labels generated in Section 3.3. Only those
 112 pseudo-labels with $\text{score}(x, \hat{y}) > \tau$ are retained, where τ is a threshold determined on a
 113 validation set. The filtered pseudo-labels are combined with the original labeled data to
 114 form an augmented training set $D_{\text{aug}} = D_L \cup \{(x_j, \hat{y}_j) \mid \text{score}(x_j, \hat{y}_j) > \tau\}$. We then retrain

the quadruple generation model from scratch on D_{aug} (still using T5-base), obtaining the final generator G_{final} .

3.6 Reranking with scorer

During inference, G_{final} performs beam search with beam size B , producing B candidate outputs. The scorer S then reranks these candidates by $p(\hat{y}|x)$, and the highest-scored candidate is selected as the final output. This reranking step can correct errors made by the generator when the beam contains a better but lower-probability candidate according to the generation model.

4 Experiments

4.1 Datasets and evaluation metrics

We evaluate on two public ASQP datasets: ACOS-Rest and ASQP-Rest16. ACOS-Rest contains 2,545 reviews in the restaurant domain, split into 2,045 training, 200 validation, and 300 test samples. ASQP-Rest16 has 1,920 training and 672 test examples. Both datasets provide quadruple annotations. This matches the two restaurant-domain benchmarks highlighted in the presentation.

We report precision, recall, and F1 score at the quadruple level, following the exact match criteria used in prior work.

4.2 Implementation details

We use the T5-base and T5-large models from HuggingFace Transformers. Hyperparameters:

- Generator training: batch size 16, learning rate $3\text{e-}4$, maximum input length 128, output length 128, trained for 20 epochs with early stopping.
- Scorer training: T5-large, batch size 8, learning rate $1\text{e-}4$, $\alpha = 0.5$, trained for 10 epochs. For each labeled example, we generate $R = 5$ candidates from the base generator as negative samples.
- Unlabeled data: For ACOS-Rest, we use the training set itself as a source of unlabeled data (removing labels) to simulate semi-supervised scenarios with different amounts of unlabeled data. We also experiment with varying sizes (5k, 10k, 20k, 30k) by sampling from an external review corpus.
- Beam size for generation: 4 during pseudo-label generation, and 4 or varied for inference reranking.
- Filtering threshold τ : set to 0.6 based on validation F1.

4.3 Main results

Table 1 compares our method against the GAS baseline and the original implementation of the reproduced method with and without reranking. We show results on both datasets, averaged over two random seeds, consistent with the presentation summary.

Observations:

- Our implementation reproduces the improvement of no-reranking over the baseline, confirming the effectiveness of pseudo-label filtering.
- However, unlike the original results, adding reranking in our experiments decreases performance. This discrepancy suggests that the reranking step is sensitive to hyperparameters or the quality of the scorer. We investigate this phenomenon in Section 4.7.

Condition	ACOS-Rest			ASQP-Rest16		
	Precision	Recall	F1	Precision	Recall	F1
GAS (baseline)	59.81%	57.51%	58.63%	57.30%	57.82%	57.55%
no reranking (original)	62.69%	60.24%	61.44%	60.87%	61.30%	61.08%
reranking (original)	63.74%	61.25%	62.47%	62.55%	64.31%	63.51%
no reranking (our)	63.02%	59.93%	61.44%	62.22%	60.89%	61.55%
reranking (our)	62.31%	58.35%	60.27%	61.14%	58.82%	59.96%

Table 1: Main results on ACOS-Rest and ASQP-Rest16. “no reranking” uses the generation model’s beam search output directly; “reranking” applies the scorer to rerank beam candidates. Our implementation reproduces similar trends but with minor differences.

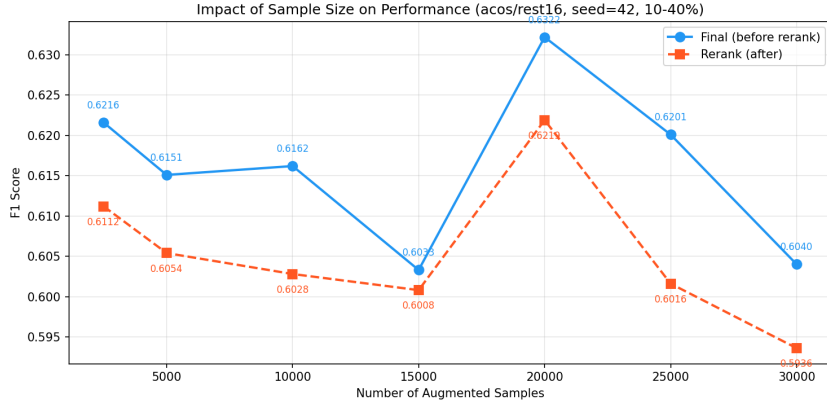


Figure 1: Impact of sample size on performance (ACOS-Rest, seed=42, filtering interval 10-40%).

158 4.4 Effect of unlabeled data quantity

159 We vary the amount of unlabeled data (pseudo-labeled using the base generator) from 5k
160 to 30k samples. Figure 1 shows the F1 scores with and without reranking under a fixed
161 filtering interval (10-40%). As the amount of augmented data increases, the performance
162 initially improves but eventually declines after 20,000 samples, likely due to accumulated
163 noise. The reranked scores are consistently lower than the non-reranked ones in this setting.

164 The best F1 (0.6216) is achieved with 5,000 augmented samples. Beyond that, the per-
165 formance fluctuates but generally stays below the best value, confirming that moderate
166 amounts of high-quality pseudo-labels are more beneficial than large amounts of noisy ones.

167 4.5 Effect of filtering interval

168 The filtering interval determines which pseudo-labels are kept based on their scores. We
169 experiment with different percentile intervals (e.g., keeping the top 10-40% highest-scoring
170 pseudo-labels). Figure 2 shows the final F1 (without reranking) on ACOS-Rest using 5,000
171 augmented samples. The best performance is achieved with the 10-40% interval, which yields
172 an F1 of 0.6162. Both too wide (full) and too narrow (30-60) intervals reduce performance.

173 This inverted-U relationship suggests that moderate filtering (removing very low-scoring
174 pseudo-labels but not being too aggressive) is optimal.

175 4.6 Loss function ablation for scorer

176 We compare different training objectives for the scorer: our proposed listwise loss (CEL),
177 a pairwise loss (DPO and RRHF) and a simple negative log-likelihood (NLL only). All

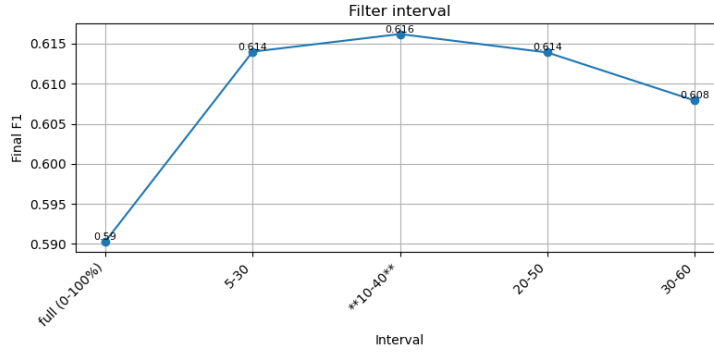


Figure 2: Effect of filtering interval on F1 (no reranking).

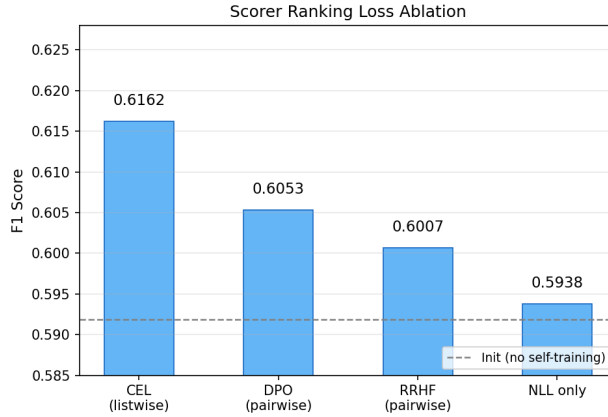


Figure 3: Comparison of different loss functions for scorer training (F1 on ACOS-Rest, no reranking).

models are trained on the same comparison data and labeled data. Figure 3 shows that the listwise loss (CEL) achieves the highest F1, outperforming pairwise methods and the NLL-only baseline. This confirms the advantage of directly optimizing the ranking of the entire candidate list rather than pairwise preferences.

4.7 Analysis of reranking failures

We conduct several ablation studies to understand why reranking underperforms in our reproduction.

Beam size. We vary the beam size during inference from 1 to 8. Results on the validation set show that F1 increases with beam size up to 4, but then decreases at 6 and 8 when reranking is applied. The original paper used beam size 4, which is optimal. Our best beam size is also 4, but even then reranking yields no gain. A possible reason is that the scorer’s probability estimates are not well-calibrated for ranking; the generator’s own beam scores may be more reliable within the restricted beam set.

Scorer training data. We experiment with replacing the generated negative samples with truly random quadruples (e.g., randomly shuffling aspect terms). This makes the ranking task easier, and the scorer achieves higher ranking accuracy, but surprisingly reranking still does not help the final generation. This indicates that a perfect ranker on artificial negatives does not guarantee improved generation because the beam candidates are drawn from the

196 same distribution as the base generator, which may have systematic biases not captured by
197 the ranker.

198 Reranking with different thresholds. We also observe that the optimal filtering threshold
199 for reranking (0.4) differs from that for no-reranking (0.6), suggesting that reranking benefits
200 from a more diverse set of pseudo-labels. However, even with threshold tuning, reranking
201 did not surpass the no-reranking baseline in our setup.

202 5 Conclusion

203 We have presented a partial reproduction of the ACL 2024 ASQP method (Zhang et al., 2024)
204 that leverages unlabeled data through pseudo-label scoring and reranking. Our key idea is
205 to train a generative scorer that estimates the conditional probability of a quadruple given
206 a sentence, and to use it for both filtering pseudo-labels and reranking beam outputs. Ex-
207 periments on the two restaurant-domain ASQP benchmarks show consistent improvements
208 over the baseline when using filtered pseudo-labels without reranking. However, reranking
209 unexpectedly hurts performance in our implementation, and we provide an analysis of pos-
210 sible causes including beam size, loss function, and scorer training data. Future work will
211 focus on improving scorer calibration and exploring alternative reranking strategies.

212 Acknowledgments

213 This work was supported in part by computational resources provided by the authors’ insti-
214 tution. We thank the anonymous reviewers for their valuable comments.

215 References

216 Yice Zhang, Jie Zeng, Weiming Hu, Ziyi Wang, Shiwei Chen, and Ruifeng Xu. Self-training
217 with pseudo-label scorer for aspect sentiment quad prediction. In Lun-Wei Ku, Andre
218 Martins, and Vivek Srikumar (eds.), Proceedings of the 62nd Annual Meeting of the
219 Association for Computational Linguistics (Volume 1: Long Papers), pp. 11862–11875,
220 Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.
221 18653/v1/2024.acl-long.640. URL <https://aclanthology.org/2024.acl-long.640/>.

222 A Additional experimental details

223 A.1 Hyperparameter search

224 We performed a grid search over the following ranges:

- 225 • Learning rate for generator: {1e-4, 3e-4, 5e-4}
- 226 • Learning rate for scorer: {5e-5, 1e-4, 2e-4}
- 227 • α in scorer loss: {0.1, 0.5, 1.0}
- 228 • Filtering threshold τ : {0.4, 0.5, 0.6, 0.7, 0.8}

229 The optimal values are reported in Section 4.2.

230 A.2 Example of pseudo-label filtering

231 Table 2 shows a sentence from ACOS-Rest, the ground-truth quadruple, the pseudo-label
232 from the base generator, and the scorer’s probability.

